Namal University, Mianwali

DEPARTMENT OF COMPUTER SCIENCES

FareShare: Ride-Sharing & Fare-Splitting System

Database Design Document

Submitted By:

Muhammad Naveed	NUM-BSCS-2024-54
Munawar Ali	NUM-BSCS-2024-60
Areeba Tahir	NUM-BSCS-2024-15

Submission Date:

14th June, 2026

Supervisor: Ms. Asiya Batool

REVISION HISTORY

Date	Version	Description	Approved by
14/06/26	V 2.0	Final submission with complete implementation.	Ms. Asiya Batool
15/05/26	V 1.0	Initial design and ERD specification.	Ms. Asiya Batool

Contents

1 PROJECT OVERVIEW **3**
- 1.1 INTRODUCTION . . . 3
- 1.2 PROBLEM STATEMENT . . . 3
- 1.3 PROJECT OBJECTIVES . . . 3
- 1.4 GITHUB REPOSITORY LINK . . . 3

2 CONCEPTUAL DATABASE DESIGN **4**
- 2.1 ENTITY . . . 4
- 2.2 DATA DICTIONARY . . . 4
- 2.3 BUSINESS RULES . . . 5
- 2.4 ENTITY RELATIONSHIP DIAGRAM . . . 5

3 LOGICAL DATABASE DESIGN **7**
- 3.1 RELATIONAL SCHEMA . . . 7
- 3.2 FUNCTIONAL DEPENDENCIES . . . 7
- 3.3 NORMALIZATION . . . 8

4 IMPLEMENTATION **9**
- 4.1 LANGUAGE/FRAMEWORK . . . 9
- 4.2 DATABASE CONNECTIVITY . . . 9
- 4.3 STORED PROCEDURES AND FUNCTIONS . . . 9
- 4.4 INTERFACES . . . 9

5 REFERENCES **12**

6 APPENDIX: AI USAGE AND PROMPTS **13**

1 PROJECT OVERVIEW

1.1 INTRODUCTION

FareShare is an innovative ride-sharing and fare-sharing platform designed specifically for the Pakistani market, focusing initially on the Mianwali region. This document outlines the detailed database design for the system, which serves as the backbone for managing users (Riders and Drivers), ride bookings, dynamic fare-splitting, safety features, and administrative oversight.

The system operates on a cash-only model and requires manual driver verification by administrators, presenting unique challenges in data consistency, relational integrity, and audit logging. The database is engineered to handle complex, real-time relationships between multiple entities, such as multiple riders sharing a single ride resource, ensuring efficient and normalized data management.

1.2 PROBLEM STATEMENT

Current ride-hailing solutions are not fully adapted to the socio-economic and trust-based environment of smaller Pakistani cities. They lack a robust mechanism for ad-hoc fare-sharing among strangers or acquaintances, which is a critical need for cost-sensitive commuters like students and daily-wage workers.

From a database perspective, this creates a significant challenge in managing a many-to-many relationship between riders and a single ride. Furthermore, a purely digital payment and verification model is not feasible. The system must rely on a cash transaction log and a manual document verification process for drivers. The core database problem is modeling these complex interactions to ensure data integrity, prevent anomalies, and support features like dynamic fare splitting and emergency contact notifications in a way that is both reliable and fully auditable.

1.3 PROJECT OBJECTIVES

General Objective: To design and implement a robust relational database that perfectly models the FareShare system's entities, their attributes, and their complex interconnections to ensure data integrity and support all functional requirements.

Specific Objectives:

- Design a conceptual model (EERD) accurately representing the supertype/subtype relationship of the User entity.
- Model the complex many-to-many relationship between Riders and Rides when fare-sharing is enabled.
- Ensure data integrity for the fare-splitting algorithm by accurately storing individual segment distances and fares.
- Create an auditable structure for driver document verification and SOS emergency alerts.

1.4 GITHUB REPOSITORY LINK

https://github.com/Naveed83067/FareShare-Ride-Sharing-System

2 CONCEPTUAL DATABASE DESIGN

2.1 ENTITY

The following table identifies and defines the core entities central to the FareShare database system.

Sr. No	Entity Name	Description
01	User	A supertype entity representing any individual with a registered account.
02	Rider	Subtype of User - searches for, books, and pays for rides.
03	Driver	Subtype of User - owns vehicle, offers rides, earns money.
04	Admin	Subtype of User - manages system, verifies drivers.
05	Vehicle	Physical vehicle owned by a Driver.
06	Ride	Core transactional entity representing a journey.
07	RideParticipant	Associative entity resolving many-to-many between Rider and Ride.
08	Payment	Cash payment transaction record.
09	DriverVerification	Tracks manual verification of driver documents by Admin.
10	SOSAlert	Emergency alert triggered during active ride.
11	EmergencyContact	Weak entity storing trusted contacts (max 3 per user).
12	Rating	Stores rating and feedback between users (max 2 per ride).

2.2 DATA DICTIONARY

This section provides detailed attribute definitions for each entity.

2.2.1 User (Supertype)

Sr. No	Name	Data Type	Constraint	Description
01	user_id	CHAR(36)	PRIMARY KEY	Unique identifier for every user
02	phone_number	VARCHAR(13)	NOT NULL, UNIQUE	Pakistan mobile number (+92 forma
03	full_name	VARCHAR(100)	NOT NULL	Full name of the user
04	user_role	ENUM	NOT NULL	Subtype discriminator (Rider, Drive
05	profile_photo	VARCHAR(255)	NULL	URL to profile picture
06	created_at	TIMESTAMP	NOT NULL	Account creation timestamp

2.2.2 Driver (Subtype)

Sr. No	Name	Data Type	Constraint	Description
01	user_id	CHAR(36)	PRIMARY KEY, FOREIGN KEY	References User superty
02	cnic_number	VARCHAR(15)	NOT NULL, UNIQUE	CNIC number
03	license_number	VARCHAR(20)	NOT NULL, UNIQUE	Valid driving license nu
04	is_verified	BOOLEAN	NOT NULL	Document verification s
05	is_online	BOOLEAN	NOT NULL	Current availability statu
06	current_location	POINT	NULL	Last known GPS coordi
07	rating_average	DECIMAL(3,2)	DEFAULT 5.00	Average rating received

2.3 BUSINESS RULES

The following business rules govern data integrity and relationships within the FareShare system:

1. A User must have exactly one role: 'Rider', 'Driver', or 'Admin' (disjoint specialization).
2. A Driver must possess exactly one registered Vehicle (1:1 relationship).
3. A Driver must be verified (is_verified = TRUE) before they can go online (is_online = TRUE) and accept rides.
4. A Ride can be regular (is_shared = FALSE) with exactly one Rider, or shared (is_shared = TRUE) with multiple Riders.
5. For a shared ride, the RideParticipant entity must record each Rider's specific segment_fare.
6. A Payment record is created for each RideParticipant (1:1 relationship) and marked is_received = TRUE only upon driver's cash confirmation.
7. An SOSAlert can only be triggered by a Rider or Driver who is an active participant in an 'In-Progress' ride.
8. An Admin must perform manual verification to change a DriverVerification record and the corresponding Driver's is_verified flag.
9. Maximum 3 emergency contacts per user.
10. Maximum 2 ratings per ride (one from rider to driver, one from driver to rider).

2.4 ENTITY RELATIONSHIP DIAGRAM

The following ERD uses Crow's Foot notation to represent the logical structure of the FareShare database. Rectangles represent entities, diamonds represent relationships, and cardinalities are shown using the Crow's Foot symbols at the ends of relationship lines.

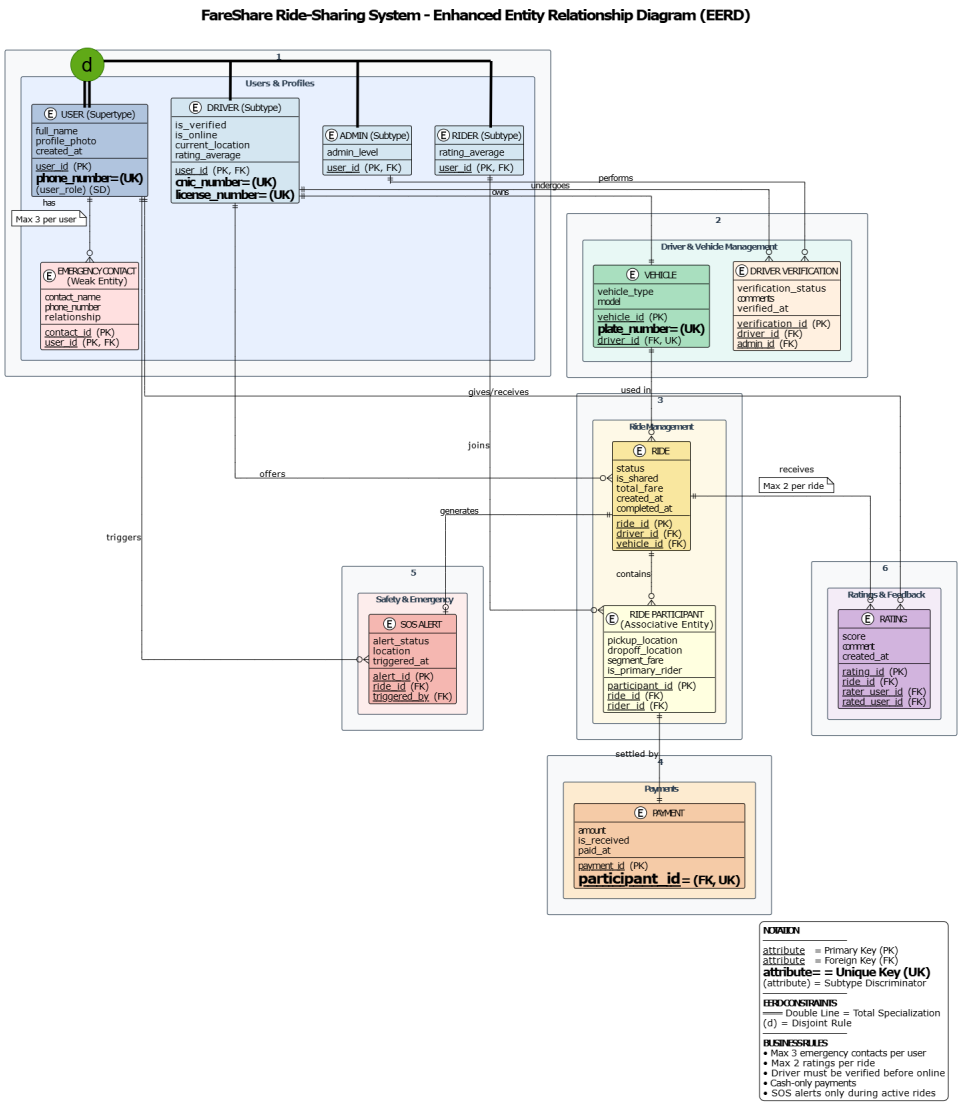

Figure 1: FareShare Entity Relationship Diagram in Crow's Foot Notation

3 LOGICAL DATABASE DESIGN

3.1 RELATIONAL SCHEMA

This section outlines the relational design of the FareShare Ride-Sharing System based on its conceptual EERD model. It demonstrates how high-level entities and relationships are systematically converted into structured database tables. Each relation is defined with appropriate keys to ensure integrity and proper linkage between data. The resulting schema is optimized for consistency, normalization, and practical database implementation.

- **USER**(user_id, phone_number, full_name, user_role, profile_photo, created_at)
- **DRIVER**(user_id, cnic_number, license_number, is_verified, is_online, current_location, rating_average)
- **RIDER**(user_id, rating_average)
- **ADMIN**(user_id, admin_level)
- **EMERGENCY_CONTACT**(contact_id, user_id, contact_name, phone_number, relationship)
- **VEHICLE**(vehicle_id, driver_id, plate_number, vehicle_type, model)
- **DRIVER_VERIFICATION**(verification_id, driver_id, admin_id, verification_status, comments, verified_at)
- **RIDE**(ride_id, driver_id, vehicle_id, status, is_shared, total_fare, created_at, completed_at)
- **RIDE_PARTICIPANT**(participant_id, ride_id, rider_id, pickup_location, dropoff_location, segment_fare, is_primary_rider)
- **PAYMENT**(payment_id, participant_id, amount, is_received, paid_at)
- **SOS_ALERT**(alert_id, ride_id, triggered_by, alert_status, location, triggered_at)
- **RATING**(rating_id, ride_id, rater_user_id, rated_user_id, score, comment, created_at)

3.2 FUNCTIONAL DEPENDENCIES

This section identifies and analyzes the functional dependencies for each relation in the FareShare Ride-Sharing System. Functional dependencies describe how attributes are logically determined by primary keys, ensuring data integrity and supporting normalization.

3.2.1 USER

- user_id $\rightarrow$ phone_number, full_name, user_role, profile_photo, created_at
- phone_number $\rightarrow$ user_id (candidate key dependency)

The USER relation represents the core entity of the system. Each user is uniquely identified by user_id, which determines all personal and account-related attributes.

3.2.2 DRIVER

- user_id $\rightarrow$ cnic_number, license_number, is_verified, is_online, current_location, rating_average

The DRIVER relation is a subtype of USER. All driver-specific attributes depend fully on user_id, ensuring that driver data cannot exist without a USER entity.

3.2.3 RIDE

- ride_id → driver_id, vehicle_id, status, is_shared, total_fare, created_at, completed_at

The RIDE relation represents the main transaction in the system. All ride details are fully dependent on ride_id.

3.3 NORMALIZATION

3.3.1 1NF (First Normal Form)

All attributes are atomic and no repeating groups exist. In the FareShare Ride-Sharing System, First Normal Form was achieved by restructuring the initial ERD into separate relational tables. Multi-valued attributes such as emergency contacts and ride participants were removed by creating dedicated relations (EMERGENCY_CONTACT and RIDE_PARTICIPANT). Additionally, subtype attributes of USER were separated into DRIVER, RIDER, and ADMIN tables to avoid mixing unrelated data. As a result, every table now contains only single-valued attributes with clearly defined primary keys, ensuring that the database fully complies with 1NF requirements.

3.3.2 2NF (Second Normal Form)

All relations have single-attribute primary keys, so no partial dependency exists. In the FareShare Ride-Sharing System, most tables use surrogate keys such as user_id, ride_id, vehicle_id, and payment_id, which ensures that every non-key attribute depends on the whole primary key.

A theoretical check was also performed for relations that could have composite keys, such as RIDE_PARTICIPANT and RATING. In both cases, attributes like pickup_location, dropoff_location, segment_fare, score, and comment depend on the full combination of identifiers rather than a partial key. Therefore, no attribute is dependent on only part of a composite key in any relation. As a result, all relations satisfy Second Normal Form (2NF) without requiring further decomposition.

3.3.3 3NF (Third Normal Form)

No transitive dependencies exist in any relation. In the FareShare Ride-Sharing System, all non-key attributes are directly dependent on their respective primary keys and do not depend on other non-key attributes. This ensures that each relation stores only logically related data and avoids redundancy across the database.

For example, attributes in USER depend only on user_id, DRIVER attributes depend only on user_id without relying on USER-derived attributes, and RIDE, PAYMENT, SOS_ALERT, and RATING relations all maintain direct dependency on their primary keys without any intermediate dependency chain.

Note: The attribute PAYMENT.amount is intentionally stored even though it can be derived from RIDE_PARTICIPANT.segment_fare. This is a justified denormalization to support financial auditing, ensure historical payment accuracy, and maintain an independent transaction record.

4 IMPLEMENTATION

4.1 LANGUAGE/FRAMEWORK

The FareShare system is implemented using a robust and modern technology stack:

- **Backend & GUI:** Python 3.10+ with Tkinter for the graphical user interface. Python was chosen for its rapid development capabilities, extensive library support (mysql-connector-python), and cross-platform compatibility.
- **Database:** MySQL 8.0, selected for its strong ACID compliance, support for spatial data types (POINT for locations), and proven reliability in production environments.
- **Database Features:** Extensive use of Stored Procedures, Triggers, and Views to enforce business logic at the database level.

4.2 DATABASE CONNECTIVITY

The GUI application connects to the MySQL database using the 'mysql-connector-python' library. The connection is managed by a dedicated 'DatabaseManager' class that handles connection pooling, query execution, and transaction management.

Connection Parameters: [language=Python, caption=Database Connection Configuration] self.connection = mysql.connector.connect(host='localhost', database='fareshare$_d b', user = ' root', password = ' Dbh83067', autocommit = False)$

All database operations are performed through Stored Procedures to ensure secure and structured data access, preventing direct SQL injection vulnerabilities.

4.3 STORED PROCEDURES AND FUNCTIONS

The following database programmability objects were developed for the project:

Object Name	Type	Description	Input Parameters
sp_register_rider	Procedure	Registers a new rider.	user_id, phone, full_nam
sp_register_driver	Procedure	Registers a new driver with vehicle.	user_id, phone, full_nam
sp_book_ride	Procedure	Creates a ride, participant, and payment.	ride_id, driver_id, vehicl
sp_complete_ride	Procedure	Marks ride as completed and payment received.	ride_id, participant_id
sp_submit_rating	Procedure	Submits a rating for a ride.	rating_id, ride_id, rater_
sp_trigger_sos	Procedure	Creates an SOS alert.	alert_id, ride_id, triggere
sp_verify_driver	Procedure	Admin verifies a driver.	verification_id, driver_id

4.4 INTERFACES

The Graphical User Interface (GUI) is designed with distinct dashboards for each user role: Rider, Driver, and Admin.

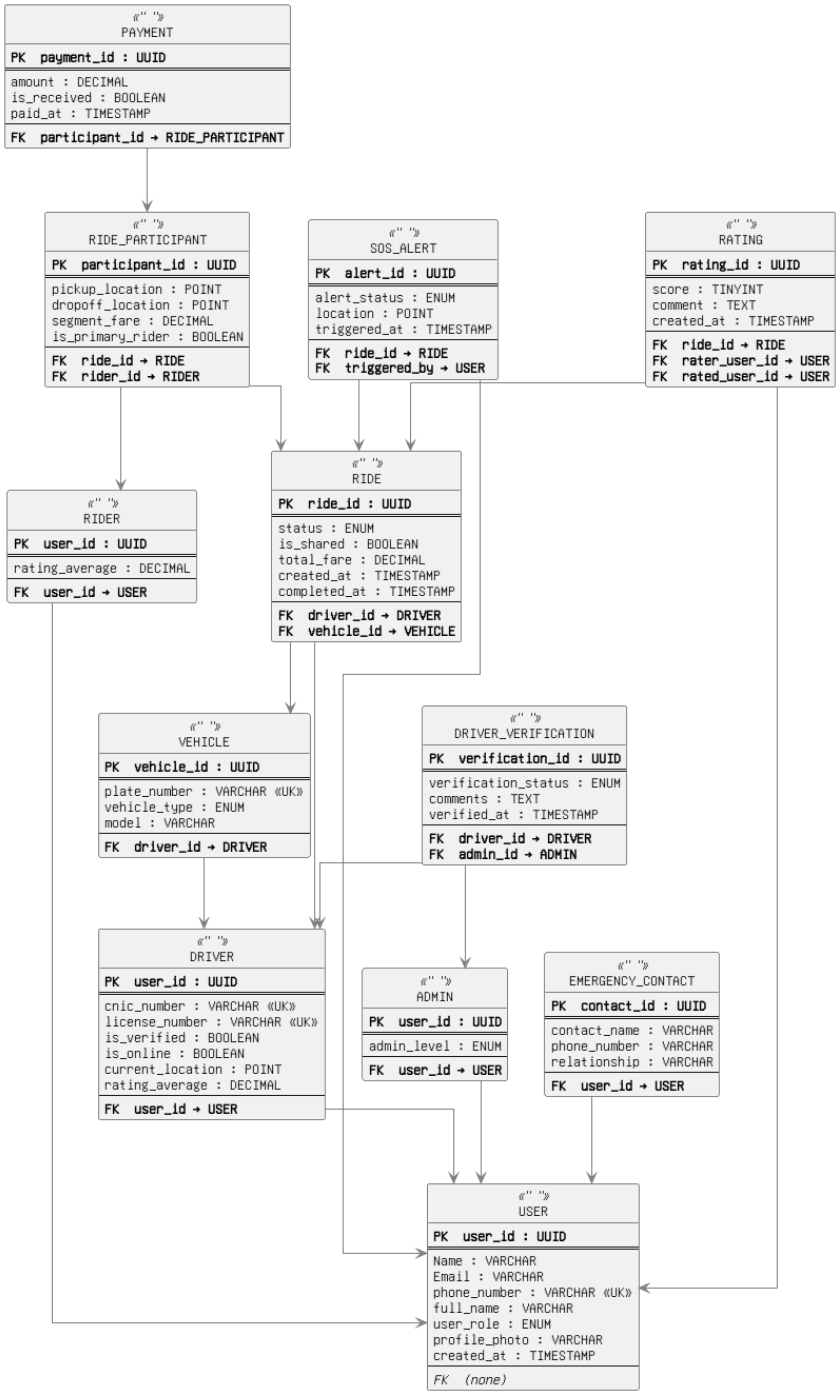

Figure 2: Database Schema Overview

Rider Dashboard: The rider interface allows users to book rides, view ride history, manage emergency contacts, and trigger SOS alerts. The main booking screen displays available drivers in real-time and provides fare estimation.

Driver Dashboard: Drivers can toggle their online/offline status, receive and respond to ride requests, view their earnings, and manage their ride history. Ride requests appear as popup notifications with a countdown timer for acceptance.

Admin Dashboard: Administrators have access to system analytics, driver verification queues,

user management, and ride monitoring. The dashboard displays key metrics such as total revenue, active rides, and pending verifications.

5 REFERENCES

[1] FareShare Software Requirements Specification (SRS), Version 2.0, Team FareShare, January 11, 2026.

[2] Project Proposal: FareShare Database System, Team FareShare, March 12, 2026.

[3] Hoffer, J. A., Ramesh, V., & Topi, H. (2019). *Modern Database Management* (13th ed.). Pearson.

[4] Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

[5] IEEE Std 830-1984. (1984). *IEEE Guide to Software Requirements Specifications*. IEEE Computer Society.

6 APPENDIX: AI USAGE AND PROMPTS

AI Tools Used:

- Google Gemini - For generating LaTeX boilerplate and ERD structure.
- DeepSeek AI - For enhancing database design, generating SQL scripts, and creating Python GUI code.

Purpose: To generate LaTeX boilerplate, Crow's Foot notation ERD structure based on project SRS, to enhance the database design document, generate DDL/DML scripts, and develop the complete Python GUI application.

Prompts Used:

Prompt	Purpose	Response Summary
"List all entities for a ride-sharing system with fare splitting in Pakistan"	Identify core entities	Received 12 entities including User supertype with Rider/Driver/Admin subtypes
"Generate Crow's Foot notation ERD with cardinalities for FareShare system"	Create ERD	Produced Crow's Foot structure with cardinalities and relationship lines
"Write complete SQL DDL for FareShare with tables, constraints, triggers, and procedures"	Generate database schema	Created comprehensive SQL script with 12 tables, 6 triggers, 7 procedures, and 4 views
"Create a Python Tkinter GUI for FareShare with rider, driver, and admin dashboards"	Develop application	Generated complete 1000+ line Python application with all required features
"Convert the data dictionary into LaTeX tables"	Format tables professionally	Generated colored tables with header styling and proper column widths

Declaration: We declare that AI tools were used solely for understanding concepts, generating templates, improving documentation quality, and accelerating development. All database design decisions, business rules, and code logic were reviewed, validated, and tested by the team.